PPO, expliqué pour de vrai

Du MDP à l'algorithme complet, avec les intuitions qui manquent dans les papiers

1. De quoi on parle, en une image

On a un **agent** (notre réseau de neurones) qui prend des décisions dans un **environnement** (le jeu OpenFront). À chaque instant t :

1. L'environnement donne à l'agent un **état** s_t (ce qu'il voit : carte, troupes, or…).
2. L'agent choisit une **action** a_t (attaquer, construire, débarquer…).
3. L'environnement renvoie une **récompense** $r_t \in \mathbb{R}$ (positif si c'est bien, négatif si c'est mal) et un nouvel état s_{t+1}.

L'agent veut **maximiser la somme des récompenses** sur toute la partie. Pour ça, il doit apprendre une *politique* — une règle qui dit « dans l'état s, prends l'action a avec probabilité tant ». PPO est juste *une façon spécifique* d'apprendre cette politique à partir d'expérience.

Ce qui rend ça dur : on n'a *pas* un prof qui dit « la bonne action en s_t était $a_t^\star$ ». On a juste la récompense, qui arrive parfois longtemps après la décision qui l'a provoquée — construire une Factory maintenant rapporte dans 100 pas. C'est le problème du *crédit différé*.

2. Les objets de base : MDP, politique, retour

2.1 Le MDP

Définition (Markov Decision Process)**.** *Un MDP est un tuple* $(\mathcal{S}, \mathcal{A}, P, r, \gamma, \rho_0)$ *où :*

— $\mathcal{S}$ *: espace des états possibles (pour nous : toutes les configs du jeu),*
— $\mathcal{A}$ *: espace des actions,*
— $P(s' \mid s, a)$ *:* dynamique *— la probabilité d'arriver en s' après avoir fait a en s,*
— $r(s, a) \in \mathbb{R}$ *: récompense immédiate,*
— $\gamma \in [0, 1)$ *: facteur d'actualisation (voir ci-dessous),*
— ρ_0 *: loi des états initiaux.*

L'adjectif Markov *signifie que P ne dépend que de (s, a), pas de l'historique : tout ce qu'il faut savoir du passé est contenu dans s_t.*

2.2 La politique

Définition (Politique)**.** *Une politique $\pi_\theta(a \mid s)$ est une distribution de probabilité sur $\mathcal{A}$ pour chaque état s. Elle est* paramétrée *par $\theta \in \mathbb{R}^d$: concrètement, θ ce sont tous les poids du réseau de neurones. Changer θ = changer le comportement de l'agent.*

Pourquoi une politique stochastique (et pas une fonction déterministe $s \mapsto a$) ? Trois raisons :

1. **Exploration** : si la politique était déterministe, elle ne découvrirait jamais une meilleure action.
2. **Gradient bien défini** : on va vouloir dériver, et la probabilité est différentiable, l'arg max non.
3. **Robustesse aux adversaires** : dans un jeu, être prédictible est une faiblesse.

2.3 Le retour

Définition (Retour actualisé)**.** *À partir du pas t, le retour est*

$$G_t \coloneqq r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k}.$$

Intuition. *Le facteur γ (typiquement $0{,}99$) dit « une récompense dans k pas vaut γ^k fois une récompense maintenant ». Avec $\gamma = 0{,}99$, une récompense dans 100 pas vaut $0{,}99^{100} \approx 0{,}37$ fois une récompense immédiate. Deux rôles :*

— ***Mathématique** : garantit que la somme converge même pour un horizon infini.*

— ***Comportemental*** *: encourage à récompenser* tôt. *Sans γ, l'agent serait indifférent entre gagner maintenant ou dans mille ans.*

2.4 L'objectif à maximiser

Le but de PPO (et de tout le RL) est de trouver θ qui maximise le retour *espéré* à partir d'un état initial :

$$J(\theta) \coloneqq \mathbb{E}_{\tau\sim\pi_\theta}\left[\sum_{t\geq 0}\gamma^t r_t\right],$$

où $\tau = (s_0, a_0, r_0, s_1, \dots)$ est une **trajectoire** tirée en roulant la politique dans l'environnement. L'espérance est sur deux sources d'aléa : (i) le choix d'actions par la politique, (ii) la dynamique P de l'environnement.

3. Les fonctions de valeur

Pour raisonner, on a besoin de savoir « combien vaut cet état ? » et « combien vaut cette action dans cet état ? ».

Définition (Fonctions de valeur)**.** *Pour une politique π_θ fixée :*

$$V^{\pi_\theta}(s) \coloneqq \mathbb{E}_{\pi_\theta}\big[G_t \mid s_t = s\big] \qquad \textit{(valeur de l'état } s\textit{)},$$
$$Q^{\pi_\theta}(s,a) \coloneqq \mathbb{E}_{\pi_\theta}\big[G_t \mid s_t = s,\, a_t = a\big] \qquad \textit{(valeur d'avoir fait } a \textit{ en } s\textit{)}.$$

*L'**avantage** compare les deux :*

$$A^{\pi_\theta}(s,a) \coloneqq Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s).$$

Intuition. *$V^{\pi_\theta}(s)$ répond à « si je joue avec ma politique à partir d'ici, qu'est-ce que je gagne en moyenne ? ». $Q^{\pi_\theta}(s,a)$ répond à « si je prends l'action a maintenant, puis je joue avec ma politique, qu'est-ce que je gagne ? ». L'avantage est la* différence *: $A > 0$ veut dire « cette action est meilleure que la moyenne de ce que je fais habituellement en s », $A < 0$ dit « cette action est pire que la moyenne ».*

Pourquoi l'avantage plutôt que Q brut ? Parce que la magnitude *de Q dépend de la position dans la partie (en fin de partie, tous les Q sont grands, tout est bon), alors que A reste autour de zéro : c'est un signal **calibré par la situation**.*

Dans notre code : le réseau sort une fonction de valeur $V_\phi(s)$ qui approxime V^{π_θ}. On n'apprend pas Q directement.

4. Comment on fait monter $J(\theta)$? Le gradient de politique

On veut faire $\theta \leftarrow \theta + \alpha\nabla_\theta J(\theta)$. Mais J est une espérance sur des trajectoires qui dépendent de θ — comment on dérive ça ?

Théorème (Théorème du gradient de politique, Sutton 1999)**.**

$$\nabla_\theta J(\theta) \;=\; \mathbb{E}_{s\sim d^{\pi_\theta},\, a\sim\pi_\theta(\cdot|s)}\big[\,\nabla_\theta \log \pi_\theta(a \mid s)\cdot Q^{\pi_\theta}(s,a)\,\big].$$

L'idée de la preuve (en 3 lignes) : écrire $J = \mathbb{E}_\tau[R(\tau)]$ puis sortir le gradient de l'espérance en utilisant le *log-trick* $\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\,\nabla_\theta \log p_\theta(\tau)$. Puisque $p_\theta(\tau) = \rho_0(s_0)\prod_t \pi_\theta(a_t|s_t)P(s_{t+1}|s_t,a_t)$, et que la dynamique P ne dépend pas de θ, seules les $\log \pi_\theta(a_t|s_t)$ survivent dans le gradient.

Intuition. *Le gradient dit : **si une action a eu un Q élevé, augmente sa log-probabilité ; si elle a eu un Q faible, diminue-la.** C'est tout. On monte la probabilité de ce qui a marché, on descend celle de ce qui n'a pas marché. Le* log *vient du changement de variable, mais pratiquement c'est un softmax avec cross-entropy pondérée par Q.*

4.1 Réduire la variance : la baseline

L'estimateur du théorème est non biaisé, mais sa *variance* est énorme (le Q peut valoir n'importe quoi en magnitude). Astuce standard : soustraire une **baseline** $b(s)$ qui ne dépend pas de a. L'espérance ne change pas :

$$\mathbb{E}_{a\sim\pi_\theta}\big[\nabla \log \pi_\theta(a|s)\, b(s)\big] = b(s)\nabla \underbrace{\sum_a \pi_\theta(a|s)}_{=1} = 0.$$

Le meilleur choix est $b(s) = V^{\pi_\theta}(s)$. Alors $Q^{\pi_\theta}(s,a) - V^{\pi_\theta}(s) = A^{\pi_\theta}(s,a)$ et

$$\boxed{\nabla_\theta J(\theta) \;=\; \mathbb{E}\big[\,\nabla_\theta \log \pi_\theta(a \mid s)\cdot A^{\pi_\theta}(s,a)\,\big].}$$

C'est la forme utilisée en pratique par tous les actor-critic.

5. Estimer l'avantage : GAE

En pratique on ne connaît pas A^{π_θ}, on a juste des trajectoires. Comment estimer $\hat{A}_t$? Deux extrêmes :

— **Monte-Carlo** : $\hat{A}_t = G_t - V_\phi(s_t)$. Non biaisé mais variance énorme (dépend de *toute* la fin d'épisode).
— **TD(0)** : $\hat{A}_t = \delta_t \coloneqq r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$. Faible variance (1 pas) mais biaisé (dépend de V_ϕ qui est inexact).

Définition (Generalized Advantage Estimation, Schulman 2016)**.**

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \;\coloneqq\; \sum_{l=0}^{\infty} (\gamma\lambda)^l\, \delta_{t+l} \;=\; \delta_t + \gamma\lambda\, \hat{A}_{t+1}^{\mathrm{GAE}}.$$

Proposition. $\lambda = 0 \Rightarrow \hat{A}_t = \delta_t$ *(TD, biais max).* $\lambda = 1 \Rightarrow \hat{A}_t = G_t - V_\phi(s_t)$ *(Monte-Carlo, variance max). Un* $\lambda \in [0{,}9;\, 0{,}98]$ *trouve un bon compromis.*

En pratique, on calcule les $\hat{A}_t$ **en remontant la trajectoire** (dernier pas d'abord), ce qui donne une récurrence en $O(T)$. Le *retour cible* pour entraîner le critique est $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$.

Enfin, on **normalise** les avantages par mini-batch : $\hat{A}_t \leftarrow (\hat{A}_t - \mu)/(\sigma + 10^{-8})$. Ça empêche les tailles de gradient d'exploser si une récompense anormale apparaît.

6. Le vrai problème : on ne peut faire qu'un seul pas de gradient

Le gradient de la section précédente est un *gradient π_θ-sur-π_θ* : il faut que l'espérance soit prise sous la politique *courante*. Si on fait un pas de gradient, θ change, et les trajectoires qu'on avait collectées sont obsolètes — elles ont été générées par l'ancienne politique.

Collecter une nouvelle trajectoire à chaque pas de gradient est **catastrophiquement lent**. On aimerait réutiliser le même batch pour plusieurs pas de SGD. Problème : comment corriger le fait que les données viennent de $\pi_{\theta_{\mathrm{old}}}$ et pas de π_θ ?

6.1 Échantillonnage préférentiel

Proposition (Importance sampling)**.** *Pour toute fonction f,*

$$\mathbb{E}_{a\sim\pi_\theta}[f(a)] \;=\; \mathbb{E}_{a\sim\pi_{\theta_{old}}}\left[\frac{\pi_\theta(a)}{\pi_{\theta_{old}}(a)}\, f(a)\right].$$

On définit le **ratio de vraisemblance**

$$r_t(\theta) \coloneqq \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\mathrm{old}}}(a_t \mid s_t)}.$$

Noter : à l'instant de la collecte, $\theta = \theta_{\mathrm{old}}$, donc $r_t = 1$. Au fur et à mesure qu'on fait des pas de gradient, r_t s'éloigne de 1.

L'objectif de substitution (ou *surrogate*) utilisé par TRPO et PPO est alors :

$$L^{\mathrm{IS}}(\theta) \;=\; \mathbb{E}_t\big[\, r_t(\theta)\, \hat{A}_t \,\big].$$

C'est un objectif qu'on peut dériver en θ sans avoir à collecter de nouvelles données, tant que π_θ reste proche de $\pi_{\theta_{\mathrm{old}}}$.

6.2 Pourquoi « proche » ?

Théorème (Borne de Kakade–Langford, 2002)**.** *Il existe $C > 0$ tel que*

$$J(\pi_\theta) \;\geq\; L^{\mathrm{IS}}(\theta) \;-\; C \cdot \max_s D_{\mathrm{KL}}\big(\pi_{\theta_{old}}(\cdot|s) \,\|\, \pi_\theta(\cdot|s)\big).$$

Intuition. *Si on améliore L^{IS} tout en restant proche de $\pi_{\theta_{\mathrm{old}}}$ (petit KL), on améliore aussi le vrai J. Si on s'éloigne trop, l'inégalité devient inutile : L^{IS} peut monter alors que J s'écroule.*

Autrement dit : l'IS corrige exactement l'écart si on reste local ; au-delà, le ratio r_t devient énorme ou minuscule, les gradients explosent, l'optimisation diverge.

Deux stratégies :

— **TRPO** (2015) : impose la contrainte $\mathbb{E}[D_{\mathrm{KL}}] \leq \delta$ *dur*, à chaque étape, et résout un problème d'optimisation quadratique. Coûteux et compliqué.
— **PPO** (2017) : approxime cette contrainte par un **clip** dans la fonction de perte. C'est tout.

7. Le cœur de PPO : l'objectif clippé

$$L^{\mathrm{CLIP}}(\theta) \;=\; \mathbb{E}_t\Big[\, \min\big(\, r_t(\theta)\, \hat{A}_t,\;\; \mathrm{clip}(r_t(\theta),\, 1-\epsilon,\, 1+\epsilon)\, \hat{A}_t \,\big) \,\Big].$$

avec typiquement $\epsilon = 0{,}2$.

C'est l'équation la plus importante du papier. Déchiffrons-la cas par cas.

7.1 Lecture géométrique

Pour un t donné, considérons $L_t = \min(r_t\hat{A}_t, \mathrm{clip}(r_t, 1-\epsilon, 1+\epsilon)\, \hat{A}_t)$ comme fonction de r_t (donc de θ).

Cas 1 : $\hat{A}_t > 0$ (l'action était meilleure que la moyenne).

— Pour $r_t \in [1-\epsilon, 1+\epsilon]$: $L_t = r_t\hat{A}_t$. Augmenter r_t augmente la perte. On *pousse* la politique à augmenter la probabilité de cette action.
— Pour $r_t > 1+\epsilon$: $L_t = (1+\epsilon)\hat{A}_t$ constant. **Plus de gradient**. On arrête de pousser.
— Pour $r_t < 1-\epsilon$: $L_t = r_t\hat{A}_t$ (le min prend la version non-clippée, qui est plus petite). On *pousse* fort pour remonter.

Résultat : on autorise à augmenter la probabilité, mais pas au-delà d'un facteur $1+\epsilon$ par rapport à $\pi_{\theta_{\mathrm{old}}}$.

Cas 2 : $\hat{A}_t < 0$ (l'action était mauvaise).

— Pour $r_t \in [1-\epsilon, 1+\epsilon]$: $L_t = r_t\hat{A}_t$. Diminuer r_t augmente la perte. On *pousse* à réduire cette action.
— Pour $r_t < 1-\epsilon$: $L_t = (1-\epsilon)\hat{A}_t$ constant. **Plus de gradient**. On arrête.
— Pour $r_t > 1+\epsilon$: $L_t = r_t\hat{A}_t$ (le min prend la version non-clippée, plus petite car $\hat{A} < 0$). On pousse fort à redescendre.

Résultat : on autorise à diminuer la probabilité, mais pas au-delà d'un facteur $1-\epsilon$.

Intuition. *Le* min *garantit qu'on prend toujours la* ***borne la plus pessimiste*** *de l'objectif : $L^{\mathrm{CLIP}} \leq L^{\mathrm{IS}}$. On optimise un* minorant*, donc l'améliorer améliore nécessairement (approximativement) le vrai J — c'est cohérent avec Kakade–Langford. Et la coupure dure du gradient quand on sort du couloir $[1-\epsilon, 1+\epsilon]$* dans le sens où on pousserait trop *fait office de trust region.*

8. La perte complète

Dans un actor-critic (notre cas), le réseau a deux têtes : la politique π_θ et le critique V_ϕ. On minimise :

$$L^{\text{PPO}}(\theta,\phi) \;=\; -\mathbb{E}_t\Big[\underbrace{L_t^{\text{CLIP}}(\theta)}_{\text{politique}} - c_1\underbrace{(V_\phi(s_t)-\hat{R}_t)^2}_{\text{critique}} + c_2\underbrace{\mathcal{H}[\pi_\theta(\cdot|s_t)]}_{\text{entropie}}\Big].$$

Trois ingrédients :

— **Politique** (L^{CLIP}) : fait monter les log-probabilités des actions à avantage positif, descendre celles à avantage négatif, avec garde-fou.
— **Critique** (MSE) : apprend V_ϕ à prédire $\hat{R}_t = \hat{A}_t + V_\phi(s_t)_{\text{old}}$. Un meilleur critique = meilleurs avantages = meilleur gradient de politique.
— **Entropie** : prime à l'incertitude. Empêche la politique de devenir quasi-déterministe trop vite, maintient l'exploration. $c_2 \in [0;\, 0{,}01]$.

Les deux têtes partagent souvent les premières couches (le *backbone*) — c'est ce qu'on fait avec le 'MultiBranchExtractor' : les 768 features sont partagées, puis deux MLP de taille 384 pour π et V.

9. L'algorithme PPO, bout à bout

Entrées : réseau π_θ et critique V_ϕ (initialisés aléatoirement), $\epsilon = 0{,}2$, $\gamma = 0{,}99$, $\lambda = 0{,}95$, N pas par collecte, K époques, mini-batch M.

Boucle principale, jusqu'à convergence :

1. **Collecte.** Avec $\theta_{\text{old}} \leftarrow \theta$, rouler $\pi_{\theta_{\text{old}}}$ dans l'environnement pendant N pas, en parallèle sur N_{env} workers. Stocker $(s_t, a_t, r_t, \log \pi_{\theta_{\text{old}}}(a_t|s_t), V_\phi(s_t))$.
2. **Avantage.** Calculer $\hat{A}_t^{\text{GAE}}$ en remontant la trajectoire, puis $\hat{R}_t = \hat{A}_t + V_\phi(s_t)$.
3. **Normaliser** les $\hat{A}_t$ par mini-batch (centre + variance).
4. **Optimisation.** Pour $k = 1, \ldots, K$ époques :
   Mélanger les N transitions, découper en mini-batchs de taille M. Pour chaque mini-batch :
   — Calculer $r_t(\theta) = \exp\big(\log \pi_\theta(a_t|s_t) - \log \pi_{\theta_{\text{old}}}(a_t|s_t)\big)$.
   — Calculer L^{CLIP}, perte critique MSE, bonus d'entropie.
   — Une étape d'Adam sur (θ, ϕ), avec clipping de la norme du gradient (typiquement 0,5).

Hyperparamètres canoniques (Schulman 2017, SB3) :
$\gamma = 0{,}99$, $\lambda = 0{,}95$, $\epsilon = 0{,}2$, $K \in \{4, \ldots, 10\}$, $c_1 = 0{,}5$, $c_2 \in [0;\, 0{,}01]$, lr Adam $= 3 \cdot 10^{-4}$, grad-norm clip $= 0{,}5$.

10. Lien avec notre code

Le mapping, pour que ce soit concret :

— π_θ et V_ϕ : les deux têtes de 'MaskablePPO' dans `train.py`. Backbone partagé = `MultiBranchExtractor` (CNN + MLP + pooling adversaires).
— **Masques d'action** : les logits des actions illégales sont mis à $-\infty$ avant le softmax ; le ratio r_t reste bien défini, le gradient aussi. C'est tout l'apport de MaskablePPO par rapport à PPO vanille.
— **N_env** : contrôlé par `-envs` ; chacun est un process Node séparé via le pont. L'astuce de `OpenFrontVecEnv` est d'envoyer les N commandes d'abord puis de collecter les N réponses — les simulations se recouvrent.
— **Récompense** : définie dans `bridge/src/env.ts`, principalement la variation de territoire normalisée par la part équitable. Les termes télescopiques annulent l'effet des allers-retours.
— **Avantage** : SB3 calcule GAE automatiquement avec $\gamma = 0{,}99$ et $\lambda = 0{,}95$.
— **Clip** : $\epsilon = 0{,}2$ par défaut dans SB3. On ne le change jamais.

11. Les intuitions à retenir

1. **Le gradient de politique dit juste : « monte la proba de ce qui a marché »**, pondéré par la qualité relative (avantage) de l'action.
2. **On a besoin d'une baseline** pour que le signal soit calibré. C'est le rôle de V_ϕ.
3. **On veut réutiliser un batch** pour plusieurs pas de gradient, donc on introduit un ratio d'importance $r_t(\theta)$. Mais ce ratio n'est valable que si la politique n'a pas trop bougé.
4. **Le clip est juste une astuce** pour approximer la contrainte « la politique n'a pas trop bougé », sans avoir à calculer de divergence KL ni résoudre de problème quadratique.
5. **GAE** est le compromis biais/variance pour estimer l'avantage à partir d'une trajectoire.
6. **La perte totale** combine politique + valeur + entropie. Les trois sont indispensables : enlever l'entropie, la politique s'effondre vite sur une action unique ; enlever le critique, l'avantage est inchiffrable.

Références : Sutton & Barto, *Reinforcement Learning : An Introduction*, 2nd ed., 2018. Schulman et al., *TRPO*, ICML 2015. Schulman et al., *GAE*, ICLR 2016. Schulman et al., *PPO*, arXiv :1707.06347, 2017. Kakade & Langford, *Approximately Optimal Approximate RL*, ICML 2002.